

MODULE 4

Syllabus

Syntax directed translation - Syntax directed definitions, S-attributed definitions, L-attributed definitions, Bottom-up evaluation of S-attributed definitions.

Syntax-Directed Translation

- Syntax-Directed Translation (SDT) is a method used in compiler design to convert source code into another form while analyzing its structure.
- It integrates syntax analysis (parsing) with semantic rules to produce intermediate code, machine code, or optimized instructions.
- In SDT, each grammar rule is linked with semantic actions that define how translation should occur.
- These actions help in tasks like evaluating expressions, checking types, generating code, and handling errors.
- SDT ensures a systematic and structured way of translating programs, allowing information to be processed bottom-up or top-down through the parse tree.
- This makes translation efficient and accurate, ensuring that every part of the input program is correctly transformed into its executable form.

- We can associate information with a language construct by attaching attributes to the grammar symbols.
- Values of the attributes are computed by semantic rules associated with the grammar productions.
- There are two notations for associating semantic rules with productions:
 - **Syntax-directed definitions(SDD)**: High level specifications for translations. They hide many implementation details and free user from having to specify explicitly the order in which translation takes place.
 - **Translation Schemes**: Low level specifications for translations. They show some implementation details and indicate the order in which semantic rules are to be evaluated.

SDD v/s SDT Scheme

SDD

- It is a context-free grammar where attributes and rules are combined and associated with grammar symbols and productions, respectively.
- Specifies the values of attributes by associating semantic rules with the productions.
- **$E \rightarrow E + T \{ E.val := E1.val + T.val \}$**
- Specifies what calculation is to be done at each production.
- Left to right evaluation.
- Used to know the value of non-terminals.

SDT

- It refers to the translation of a string into an array of actions. This is done by adding an action to a rule of context-free grammar. It is a type of compiler interpretation.
- Embeds program fragments (also called semantic actions) within production bodies.
- **$E \rightarrow E + T \{ \text{print}(' + '); \}$**
- Specifies what calculation is to be done at each production and at what time they must be done.
- Left to right evaluation.
- Used to generate Intermediate Code.

- With both Syntax Directed Translation and Translation Schemes,
 - We parse the input token stream,
 - Build the parse tree and then
 - Traverse the tree as needed to evaluate the semantic rules at the parse tree nodes.
- Evaluation of semantic rules may
 - may generate code,
 - save information in the symbol table,
 - issue error messages
 - Perform semantic check, etc.

input string -> parse tree -> dependency graph -> evaluation order for semantic rules

- In some cases we do not have to follow this outline literally. We may evaluate the rules during parsing itself so as to improve compile-time efficiency.

Syntax Directed Definitions(SDD)

- SDD is a context-free grammars in which:
 1. Grammar symbols have an associated set of Attributes;
 2. Productions are associated with Semantic Rules for computing the values of attributes.
- Attributes may be of many kinds: strings, numbers, types, memory location etc.
- The value of an attribute at a parse-tree node is defined by a semantic rule associated with the production used at that node.
- Eg: If X is a symbol and a is one of its attributes then we write X.a to denote the value of a at a particular parse tree node labeled X.

Ex: $E \rightarrow E + T \{ E.value = E.value + T.value \}$

- The right part of the CFG contains the semantic rules that specify how the grammar should be interpreted. Here, the values of non-terminals E and T are added together and the result is copied to the non-terminal E.

- There are two kinds of attributes:
 - **Synthesized attributes**
 - The value of a synthesized attribute at a node N is computed from the values of attributes at the children of node N.
 - **Inherited attributes**
 - The value of an inherited attribute is computed from the values of attributes at the siblings and parent of that node.
- Semantic rules set up dependencies between attributes that will be represented by a graph called dependency graph.
- From dependency graph we derive an evaluation order for the semantic rules.
- A parse tree showing the values of attributes at each node is called *annotated parse tree*.
- The process of computing the attribute values at the nodes is called *annotating or decorating the parse tree*.

Syntax-directed definition of a Simple desk calculator:

This definition associates an integer-valued **synthesized attribute** called **val** with each of nonterminal.

Production

$L \rightarrow E_n$

$E \rightarrow E_1 + T$

$E \rightarrow T$

$T \rightarrow T_1 * F$

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow \text{digit}$

Semantic Rules

$\text{print}(E.\text{val})$

$E.\text{val} := E_1.\text{val} + T.\text{val}$

$E.\text{val} := T.\text{val}$

$T.\text{val} := T_1.\text{val} * F.\text{val}$

$T.\text{val} := F.\text{val}$

$F.\text{val} := E.\text{val}$

$F.\text{val} := \text{digit}.\text{lexval}$

- In the above example, for each E, T and F production, the semantic rule computes the value of attribute val for the nonterminal on the left side from the values of val from the nonterminals on the right side.
- The rule associated with the production $L \rightarrow E_n$ for the starting Nonterminal L is just a procedure that prints as output the value of arithmetic expression generated by E.
- In a SDD, terminals are assumed to have synthesized attributes only(i.e. No inherited attributes)
- There are no semantic rules for computing the value of an attribute for a terminal. Values for attributes of terminals are usually supplied by Lexical Analyser.
 - Thus, the token digit has an attribute lexval whose value is assumed to be supplied by the lexical analyser.

S-Attributed Definition

- A syntax-directed definition that uses only synthesized attributes is said to be a S-attributed definition.
- Semantic rules in a S-attributed definition can be evaluated or annotated by a **bottom up parse of the tree** i.e. from the leaves to the root.

Example of S-attributed SDD

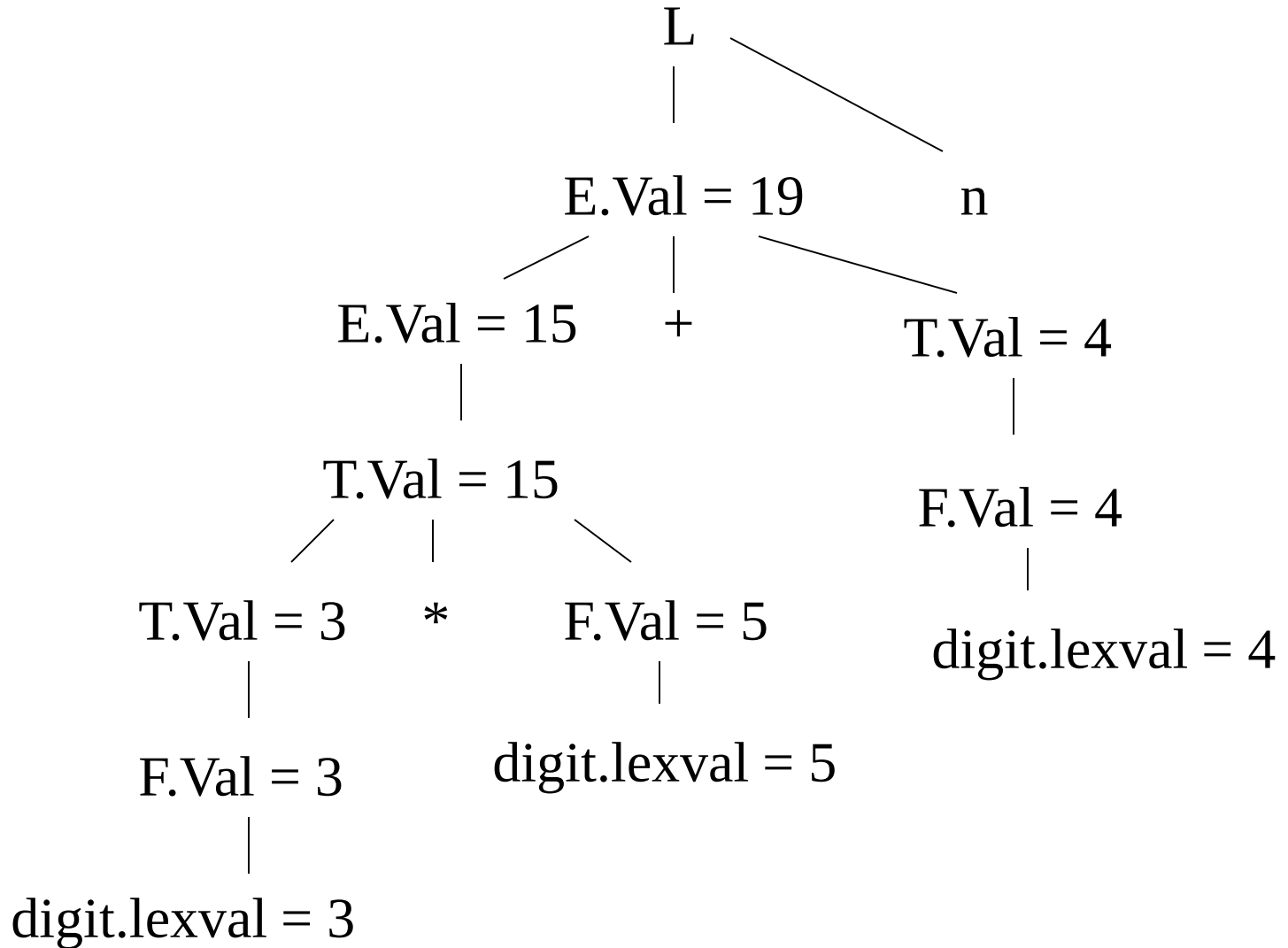
Production

- 1) $L \rightarrow E n$
- 2) $E \rightarrow E1 + T$
- 3) $E \rightarrow T$
- 4) $T \rightarrow T1 * F$
- 5) $T \rightarrow F$
- 6) $F \rightarrow (E)$
- 7) $F \rightarrow \text{digit}$

Semantic Rules

- $L.val = E.val$
 $E.val = E1.val + T.val$
 $E.val = T.val$
 $T.val = T1.val * F.val$
 $T.val = F.val$
 $F.val = E.val$
 $F.val = \text{digit.lexval}$

Annotated parse tree for $3*5+4n$ using the above grammar



- ***Inherited Attributes***

- An inherited attribute is one whose value at a node in a parse tree is defined in terms of attributes at the parent and/or siblings of that node.

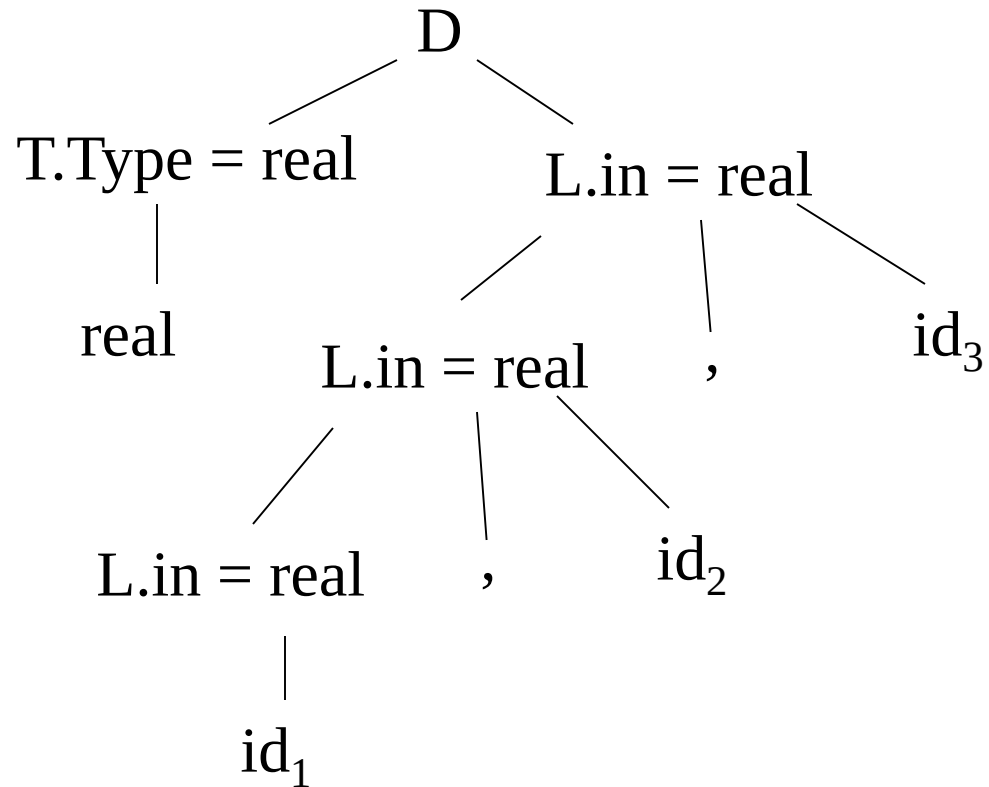
Example

- Using inherited attributes to parse a declaration and add type information to the symbol table.

Production	Semantic Rules
$D \rightarrow TL$	<u>L.in</u> := <u>T.type</u>
$T \rightarrow \text{int}$	<u>T.Type</u> := integer
$T \rightarrow \text{real}$	T.Type := real
$L \rightarrow L_1, \text{id}$	<u>L₁.in</u> := <u>L.in</u> <u>addtype</u> (<u>id.entry</u> , <u>L.in</u>)
$L \rightarrow \text{id}$	<u>addtype</u> (<u>id.entry</u> , <u>L.in</u>)

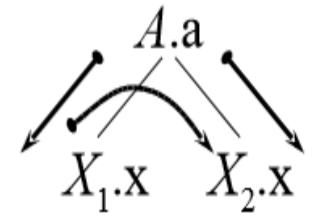
- The nonterminal T has a synthesized attribute type, whose value is determined by the keyword in declaration.
- The semantic rule $L.in := T.type$ associated with production $D \rightarrow TL$, sets inherited attribute L.in to the type in the declaration.
- The rules then pass this type down the parse tree using the inherited attribute L.in
- Rules associated with the production for L call procedure addtype to add type of each identifier to its entry in symbol table (pointed to by attribute entry)

Annotated Parse Tree for sentence: real id₁, id₂, id₃



L-Attributed Definition

- A syntax-directed definition is L-attributed if each inherited attribute of X_j on the right side of $A \rightarrow X_1 X_2 \dots X_n$ depends only on
 - the attributes of the symbols $X_1, X_2, \dots, X_{j-1}$
 - the inherited attributes of A

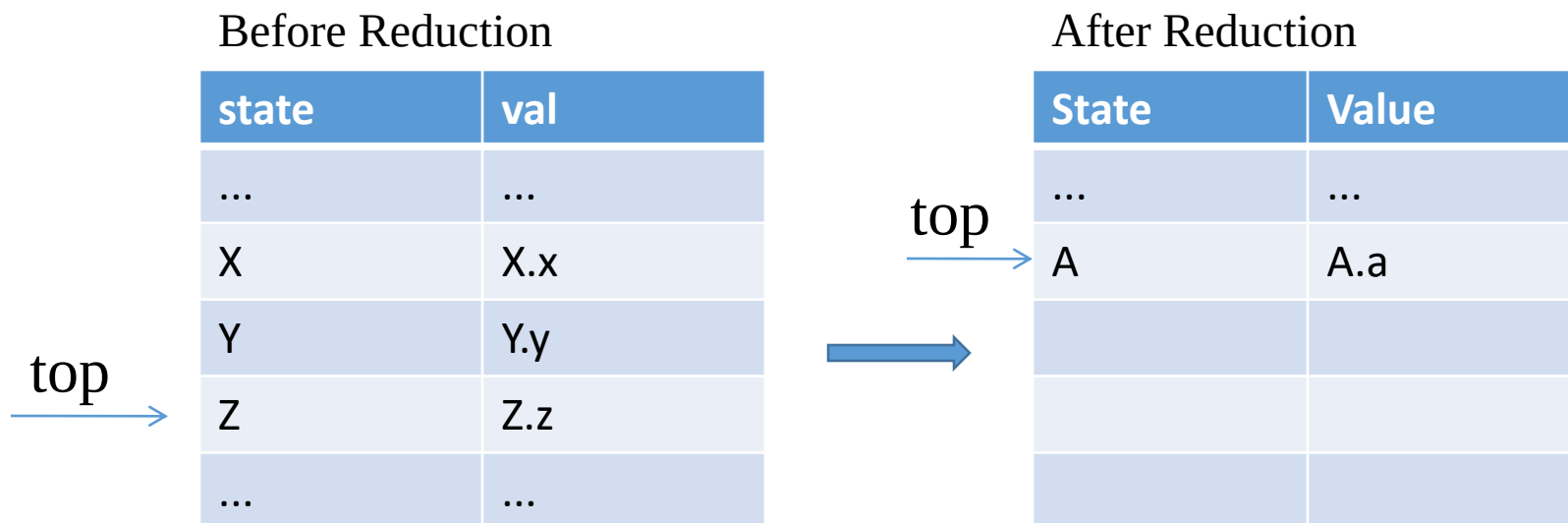


- L-attributed definitions allow for a natural order of evaluating attributes:
depth-first and left to right.
- All Attributes gets its value from its parent or from its left siblings only.
- Eg: $A \rightarrow XYZ$
 - $Y.s = A.s$ (from parent – hence correct)
 - $Y.s = X.s$ (from left sibling – hence correct)
 - $Y.s = Z.s$ (from right sibling – hence wrong)

Bottom-up Evaluation of S-Attributed Definitions

- Synthesized attributes can be evaluated by a bottom-up parser(or LR parser) as the input is being parsed.
- The parser keeps track of values of the synthesized attributes associated with the grammar symbols on its stack.
- Whenever a reduction is made, the values of the new synthesized attributes are computed from the attributes appearing on the stack.

- The stack is implemented as a pair of state and value.
- For example, for the production rule, $A \rightarrow XYZ$ and the semantic rule $A.a = f(X.x, Y.y, Z.z)$, the stack with a field for synthesized attributes are



- The current top of stack is indicated by pointer top.

Bottom-up evaluation of S-attributed definitions

- Suppose semantic rule
$$A.a = f(X.x, Y.y, Z.z)$$
is associated with production
$$A \rightarrow XYZ$$
- Before reducing XYZ to A , value of Z is in $\text{val}(\text{top})$, value of Y is in $\text{val}(\text{top}-1)$ and value of X is in $\text{val}(\text{top}-2)$
- If symbol has no attribute then the entry is undefined
- After the reduction, top is decremented by 2 and state covering A is put in $\text{val}(\text{top})$

PRODUCTION	CODE FRAGMENT
$L \rightarrow E \mathbf{n}$	<i>print</i> (<i>val</i> [<i>top</i>])
$E \rightarrow E_1 + T$	<i>val</i> [<i>ntop</i>] := <i>val</i> [<i>top</i> - 2] + <i>val</i> [<i>top</i>]
$E \rightarrow T$	
$T \rightarrow T_1 * F$	<i>val</i> [<i>ntop</i>] := <i>val</i> [<i>top</i> - 2] × <i>val</i> [<i>top</i>]
$T \rightarrow F$	
$F \rightarrow (E)$	<i>val</i> [<i>ntop</i>] := <i>val</i> [<i>top</i> - 1]
$F \rightarrow \mathbf{digit}$	

Fig. 5.16. Implementation of a desk calculator with an LR parser.

INPUT	state	val	PRODUCTION USED
3*5+4 n	-	-	
*5+4 n	3	3	
*5+4 n	F	3	$F \rightarrow \text{digit}$
*5+4 n	T	3	$T \rightarrow F$
5+4 n	T *	3 -	
+4 n	T * 5	3 - 5	
+4 n	T * F	3 - 5	$F \rightarrow \text{digit}$
+4 n	T	15	$T \rightarrow T * F$
+4 n	E	15	$E \rightarrow T$
4 n	E +	15 -	
n	E + 4	15 - 4	
n	E + F	15 - 4	$F \rightarrow \text{digit}$
n	E + T	15 - 4	$T \rightarrow F$
n	E	19	$E \rightarrow E + T$
	E n	19 -	
	L	19	$L \rightarrow E n$

Fig. 5.17. Moves made by translator on input $3*5+4n$.

- Show Simulation of bottom up evaluation of the expression $1+7*4n$ using SDD given before.

Input	State	value	Production rule
1+7*4n	—	—	
+7*4n	1	1	
+7*4n	F	1	F->digit
+7*4n	T	1	T->F
+7*4n	E	1	E->T
7*4n	E +	1 _	
*4n	E + 7	1 _ 7	
*4n	E + F	1 _ 7	F->digit
*4n	E + T	1 _ 7	T->F
4n	E + T *	1 _ 7	
n	E + T * 4	1 _ 7 _ 4	
n	E + T * F	1 _ 7 _ 4	F->digit
n	E + T	1 _ 28	T->T*F
n	E	29	E->E+T
—	E n	29	
—	L	29	L->E n

Translation Schemes

- In a syntax-directed definition, we do not say anything about the evaluation times of the semantic rules (when the semantic rules associated with a production should be evaluated).
- Translation schemes describe the order and timing of attribute computation.
- A **translation scheme** is a context-free grammar in which:
 - attributes are associated with the grammar symbols and
 - semantic actions enclosed between braces $\{\}$ are inserted within the right sides of productions.

Ex: $A \rightarrow \{ \dots \} X \{ \dots \} Y \{ \dots \}$



Semantic Actions

- Translation scheme is a useful notation for specifying translation during parsing. It can have both synthesized and inherited attributes.
- If our syntax-directed definition is S-attributed, the construction of the corresponding translation scheme will be simple.
- Each associated semantic rule in a S-attributed syntax-directed definition will be inserted as a semantic action into the end of the right side of the associated production.

Production	Semantic Rule	
$E \rightarrow E1 + T$	$E.val = E1.val + T.val$	a production of a syntax directed definition

↓

$E \rightarrow E1 + T \{ E.val = E1.val + T.val \}$	the production of the translation scheme
corresponding	

A Translation Scheme Example

- A simple translation scheme that converts infix expressions to the corresponding postfix expressions

$E \rightarrow E + T \{ \text{print } "+" \}$

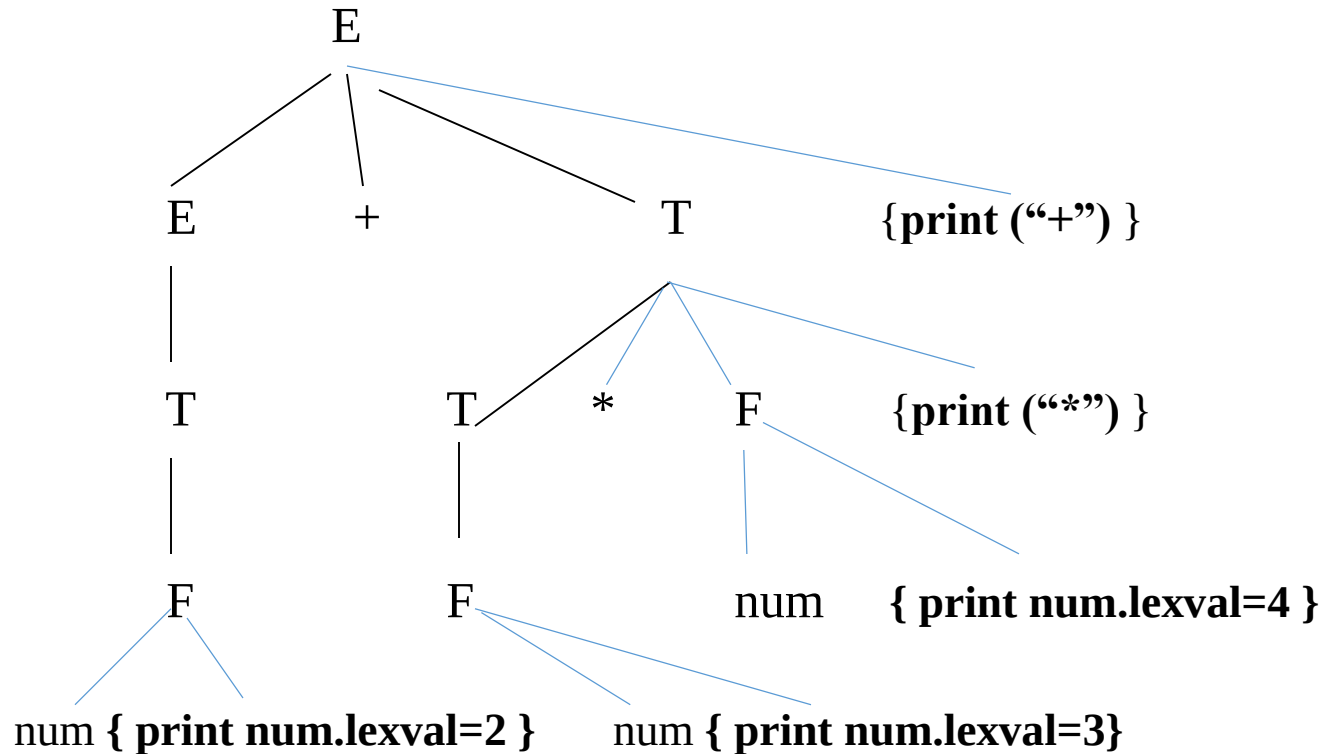
$E \rightarrow T$

$T \rightarrow T * F \{ \text{print } "*" \}$

$T \rightarrow F$

$F \rightarrow \text{num} \{ \text{print num.lexval} \}$

Convert $2+3*4$ into postfix notation using above SDT



- The depth first traversal of the parse tree (executing the semantic actions in that order) will produce the postfix representation of the infix expression which is $234*+$

Inherited Attributes in Translation Schemes

- If a translation scheme has to contain both synthesized and inherited attributes, we have to observe the following rules to ensure that the attribute value is available when an action refers to it.
 1. An inherited attribute of a symbol on the right side of a production must be computed in a semantic action before that symbol.
 2. A semantic action must not refer to a synthesized attribute of a symbol to the right of that semantic action.
 3. A synthesized attribute for the non-terminal on the left can only be computed after all attributes it references have been computed (we normally put this semantic action at the end of the right side of the production).
- With a L-attributed syntax-directed definition, it is always possible to construct a corresponding translation scheme which satisfies these three conditions (This may not be possible for a general syntax-directed translation).

Q) Construct Translation scheme for SDD with inherited attributes given below.

Production	Semantic Rules
$D \rightarrow TL$	$L.in := T.type$
$T \rightarrow \text{int}$	$T.Type := \text{integer}$
$T \rightarrow \text{real}$	$T.Type := \text{real}$
$L \rightarrow L_1, \text{id}$	$L_1.in := L.in$ $\text{addtype}(\text{id.entry}, L.in)$
$L \rightarrow \text{id}$	$\text{addtype}(\text{id.entry}, L.in)$

$$D \rightarrow T \{L.in = T.type\} L$$
$$T \rightarrow \mathbf{int} \{ T.type = \text{integer} \}$$
$$T \rightarrow \mathbf{real} \{ T.type = \text{real} \}$$
$$L \rightarrow \{L1.in = L.in\} L1, \mathbf{id} \{\text{addtype}(\mathbf{id}.entry, L.in)\}$$
$$L \rightarrow \mathbf{id} \{\text{addtype}(\mathbf{id}.entry, L.in)\}$$

- This is a translation scheme for an L-attributed definitions

Comparison b/w S-Attributed and L-Attributed SDD

S-Attributed	L-Attributed
Uses only Synthesized attributes.	Uses both synthesized and inherited attributes. Each inherited attribute is restricted to inherit either from parent or from left siblings only.
Semantic actions are placed at right end of production. Eg: $A \rightarrow XYZ \{ \}$	Semantic actions are placed anywhere on RHS of production. Eg: $A \rightarrow \{ \} XY$ $A \rightarrow X \{ \} Y$ $A \rightarrow XY \{ \}$
Attributes are evaluated during bottom-up parsing.	Attributes are evaluated by traversing parse tree depth first, left to right.

Tutorial

17. (a) Design a Syntax Directed Translator(SDT) for the arithmetic expression $(4 * 7 + 19) * 2$ and draw an annotated parse tree for the same. (8)

- (b) Consider the grammar with following translation rules and E as the start symbol (6)

$$E \rightarrow E1 \# T \{E.value = E1.value \times T.value ; \}$$
$$| T \{E.value = T.value ; \}$$
$$T \rightarrow T1 \& F \{ T.value = T1.value + F.value ; \}$$
$$| F \{ T.value = F.value ; \}$$
$$F \rightarrow \text{num} \{ F.value = \text{num}.lvalue ; \}$$

Compute E.value for the root of the parse tree for the expression

$2 \# 3 \& 5 \# 6 \& 7$

18. (a) Write Syntax Directed Translator (SDT) and parse tree for infix to postfix translation of an expression. (8)